Clarifying range adaptor objects

Document #:	P2281R1
Date:	2021-02-19
Project:	Programming Language C++
Audience:	LWG

Reply-to:	Tim Song <t.canens.cpp@gmail.com>

1 Abstract

This paper proposes resolutions for [LWG3509] and [LWG3510].

2 Revision history

— R1: Incorporate LWG review feedback.

3 Discussion

The wording below clarifies that the partial application performed by range adaptor objects is essentially identical to that performed by `bind_front`. (Indeed, it is effectively a limited version of `bind_back`.) In particular, this means that the bound arguments are captured by copy or move, and never by reference. Invocation of the pipeline then either copies or moves the bound entities, depending on the value category of the pipeline.

In other words,

```
auto c = /* some range */;
auto f = /* expensive-to-copy function object */;
c | transform(f); // copies f and then move it into the view

auto t = transform(f);  // copies f
c | t;  // copies f again from t
c | std::move(t); // moves f from t
```

For all but one range adaptor in the standard library, the bound arguments are expected to be either function objects (which are expected to be cheap to copy and are generally copied freely) or integer-like types (which should be cheap to copy).

`views::split`, where the pattern can be a range, is an interesting case for two reasons. (The pattern can be an element as well, but that is not particularly interesting.)

— If the pattern is a C array, the decay-copy makes it into a pointer. As a result, the `split_view` construction might be ill-formed if the user actually meant to use the array as a range. For the most

common case of string literals, using the array (including the terminating null character) as the pattern is likely unintended.

— If the pattern is an lvalue non-view range, the copy can make it a non-viewable range when the closure object is used as an rvalue:

```
std::string s = "hello", s1 = "l";
views::split(s, s1);    // OK
s | views::split(s1);   // ill-formed under this wording; copy of s1 forwarded as
        rvalue
auto adapt = views::split(s1);
s | adapt;              // OK
```

There does not appear to be a way to avoid this without making the following function template (which demonstrates an expected use of adaptor pipelines) always return something that holds a dangling reference:

```
template<class Pattern>
auto split_into_strings(Pattern p) {
  return views::split(p) | views::transform([](auto r){
    return std::string(r.begin(), ranges::next(r.begin(), r.end()));
  });
}
```

In both cases, the workaround is to wrap the pattern in `views::all`, which also clearly signifies the reference semantics of the capture. Having a compile-time error, while perhaps less than ideal, seems to be preferable to silent dangling.

As range adaptor objects are customization point objects, and the use of `bind_front`-like semantics means that they will be copied and invoked as non-const lvalues and possibly-const rvalues, this wording also resolves [LWG3510] by clarifying that customization point objects are invocable regardless of value category or cv-qualification.

§ 4 Wording

This wording is relative to [N4878].

— Edit 16.3.3.3.6 [customization.point.object] as indicated:

3 All instances of a specific customization point object type shall be equal (18.2 [concepts.equality]). The effects of invoking different instances of a specific customization point object type on the same arguments are equivalent.

4 The type `T` of a customization point object, ignoring cv-qualifiers, shall model `invocable<T&, Args...>`, `invocable<const T&, Args...>`, `invocable<T, Args...>`, and `invocable<const T, Args...>` (18.7.2 [concept.invocable]) when the types in `Args...` meet the requirements specified in that customization point object's definition. When the types of `Args...` do not meet the customization point object's requirements, `T` shall not have a function call operator that participates in overload resolution.

5 For a given customization point object `o`, let `p` be a variable initialized as if by `auto p = o;`. Then for any sequence of arguments `args...`, the following expressions have effects equivalent to `o(args...)`:

— `p(args...)`

— `as_const(p)(args...)`

— `std::move(p)(args...)`

— `std::move(as_const(p))(args...)`

— Edit 24.7.2 [range.adaptor.object] as indicated:

1 A *range adaptor closure object* is a unary function object that accepts a `viewable_range` argument and returns a `view`. For a range adaptor closure object `C` and an expression `R` such that `decltype((R))` models `viewable_range`, the following expressions are equivalent and yield a `view`:

```
C(R)
R | C
```

Given an additional range adaptor closure object `D`, the expression `C | D` is well-formed and produces another range adaptor closure object `E`. such that the following two expressions are equivalent:

~~`R | C | D`~~
~~`R | (C | D)`~~

`E` is a perfect forwarding call wrapper (20.14.4 [func.require]) with the following properties:

— Its target object is an object `d` of type `decay_t<decltype((D))>` direct-non-list-initialized with `D`.

— It has one bound argument entity, an object `c` of type `decay_t<decltype((C))>` direct-non-list-initialized with `C`.

— Its call pattern is `d(c(arg))`, where `arg` is the argument used in a function call expression of `E`.

The expression `C | D` is well-formed if and only if the initializations of the state entities of `E` are all well-formed.

2 A *range adaptor object* is a customization point object (16.3.3.3.6 [customization.point.object]) that accepts a `viewable_range` as its first argument and returns a `view`.

3 If a range adaptor object accepts only one argument, then it is a range adaptor closure object.

4 ~~If a range adaptor object accepts more than one argument, then the following expressions are equivalent:~~

~~`adaptor(range, args...)`~~
~~`adaptor(args...)(range)`~~
~~`range | adaptor(args...)`~~

~~In this case, `adaptor(args...)` is a range adaptor closure object.~~

4 If a range adaptor object `adaptor` accepts more than one argument, then let `range` be an expression such that `decltype((range))` models `viewable_range`, let `args...` be arguments such that `adaptor(range, args...)` is a well-formed expression as specified in the rest of this subclause (24.7 [range.adaptors]), and let `BoundArgs` be a pack that denotes `decay_t<decltype((args))>...`. The expression `adaptor(args...)` produces a range adaptor closure object `f` that is a perfect forwarding call wrapper with the following properties:

- Its target object is a copy of `adaptor`.
- Its bound argument entities `bound_args` consist of objects of types `BoundArgs...` direct-non-list-initialized with `std::forward<decltype((args))>(args)...`, respectively.
- Its call pattern is `adaptor(r, bound_args...)`, where `r` is the argument used in a function call expression of `f`.

The expression `adaptor(args...)` is well-formed if and only if the initialization of the bound argument entities of the result, as specified above, are all well-formed.

§ 5 References

[LWG3509] Tim Song. Range adaptor objects are underspecified.
https://wg21.link/lwg3509

[LWG3510] Tim Song. Customization point objects should be invocable as non-const too.
https://wg21.link/lwg3510

[N4878] Thomas Köppe. 2020-12-15. Working Draft, Standard for Programming Language C++.
https://wg21.link/n4878